

Servidor HTTP

Linguagem de Programação III

O que muda agora?

- Antes: código executava no console
 - Sem interação externa
- Agora: sistema responde requisições
 - Usuário acessa via navegador
 - Servidor responde dinamicamente
- Isso caracteriza uma API

O que é HTTP?

- Protocolo de comunicação web
- Base da internet moderna
- Define como cliente e servidor se comunicam
- Usado por navegadores e APIs
- Funciona em requisição e resposta
- Opera sobre TCP/IP

Cliente vs Servidor

- Cliente: faz a requisição
- Exemplo: navegador, app
- Servidor: processa a requisição
- Retorna uma resposta
- Comunicação ocorre via HTTP
- Base de sistemas web

Fluxo completo

- Cliente envia requisição
- Servidor recebe requisição
- Servidor processa lógica
- Servidor consulta dados
- Servidor retorna resposta
- Cliente exibe resultado

Criando servidor HTTP (completo)

```
import http from 'http';

const server = http.createServer((req, res) => {

  res.end('Servidor funcionando');

});

server.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

Explicação do código

- http: módulo nativo do Node.js
- createServer: cria servidor
- req: dados da requisição
- res: resposta ao cliente
- res.end(): finaliza resposta
- listen: inicia servidor

Testando o servidor

- Executar node app.js
- Abrir navegador
- Acessar <http://localhost:3000>
- Verificar resposta exibida
- Confirmar funcionamento
- Servidor ativo localmente

Visualizando requisição

- `console.log(req.url);`
- `console.log(req.method);`

Interpretando req

- req.url: caminho acessado
- req.method: tipo da requisição
- GET: leitura de dados
- POST: envio de dados
- PUT: atualização
- DELETE: remoção

Criando rotas (completo)

```
if (req.url === '/usuarios') {  
  
    res.end('Lista de usuários');  
  
} else if (req.url === '/produtos') {  
  
    res.end('Lista de produtos');  
  
} else {  
  
    res.statusCode = 404;  
    res.end('Rota não encontrada');  
  
}
```

Rotas manuais

- Criadas com if/else
- Comparação de URL
- Cada rota retorna resposta
- Simples e didático
- Sem frameworks

Retornando JSON (completo)

```
if (req.url === '/usuarios') {  
  
  const usuarios = [  
    { id: 1, nome: 'Ana' },  
    { id: 2, nome: 'Carlos' }  
  ];  
  
  res.setHeader('Content-Type', 'application/json');  
  res.end(JSON.stringify(usuarios));  
  
}
```

JSON em APIs

- Formato padrão de dados
- Usado em APIs REST
- Baseado em objetos JavaScript
- Leve e fácil de ler
- Compatível com frontend
- Requer conversão para string

Content-Type

- Define tipo da resposta
- application/json para APIs
- Necessário para frontend
- Ajuda navegador interpretar
- Boa prática obrigatória
- Sempre usar em JSON

Integração com backend

- Reutilizar código anterior
- Controller recebe requisição
- Service processa lógica
- Repository acessa dados
- Separação de responsabilidades
- Arquitetura organizada

Controller adaptado (completo)

```
export async function buscarUsuarioController(id, res) {  
  
  try {  
  
    const usuario = await obterUsuario(id);  
  
    res.setHeader('Content-Type', 'application/json');  
    res.end(JSON.stringify(usuario));  
  
  } catch (erro) {  
  
    res.statusCode = 404;  
    res.end('Usuário não encontrado');  
  
  }  
  
}
```

Servidor integrado (completo)

```
import http from 'http';
import { buscarUsuarioController } from './userController.js';

const server = http.createServer((req, res) => {

  if (req.url === '/usuarios/1') {

    buscarUsuarioController(1, res);

  } else {

    res.statusCode = 404;
    res.end('Rota não encontrada');

  }

});

server.listen(3000);
```

Fluxo final da aplicação

- Cliente faz requisição HTTP
- Servidor recebe requisição
- Controller decide ação
- Service aplica regras
- Repository busca dados
- Resposta retorna ao cliente

Erros comuns

- Esquecer `res.end()`
- Não usar `JSON.stringify`
- Erro na URL da rota
- Não definir `statusCode`
- Não usar `Content-Type`
- Confundir `req` e `res`